Exploratory Analysis of Rainfall Data in India for Agriculture

Project Documentation

Team ID : LTVIP2026TMIDS71741
Team Size : 4
Team Leader : Bhavana Rasupalli
Team member : Bolla Chakradhar Reddy
Team member : Brahma Koteswarao Pillakatupu
Team member : Bulla Vijay Kumar

Abstract / Project Overview

The Exploratory Analysis of Rainfall Data in India for Agriculture aims to analyze historical rainfall data to uncover patterns, trends, and relationships that can support agricultural decision-making. Using data visualization and statistical methods, the study provides insights into rainfall distribution, helping farmers and policymakers plan irrigation, crop selection, and resource allocation more effectively.

Objectives

The key objectives of this project are:

- To understand the rainfall distribution across different regions and time periods in India.
- To identify missing data and clean the dataset for accurate analysis.
- To visualize rainfall trends and relationships using graphs and statistical plots.
- To extract insights that can help in agricultural planning and policy formulation.

Technologies Used (Python, Libraries, etc.)

The project utilizes Python programming language along with several essential libraries for data handling and visualization:

- Python: Core programming language used for analysis.
- NumPy: For numerical computations and array manipulation.
- Pandas: For data cleaning, transformation, and exploration.
- Matplotlib: For basic plotting and charting.
- Seaborn: For advanced and aesthetically appealing statistical visualizations.
- Scikit-learn: For preprocessing tasks such as scaling and encoding.

Dataset Description

The dataset consists of historical rainfall observations collected from multiple regions in India. It includes attributes such as temperature, humidity, wind speed, evaporation rate, and rainfall amount. The target variable, typically 'RainTomorrow', indicates whether rainfall is expected on the following day. The dataset provides both numerical and categorical features that need to be preprocessed before visualization.

Data Preprocessing

Data preprocessing is a crucial step to ensure data quality and consistency before visualization and model building. The main steps include importing necessary libraries, handling missing values, encoding categorical features, and scaling numerical features.

Importing Libraries

The primary Python libraries used in this project include Pandas, NumPy, Matplotlib, and Seaborn. These libraries support data manipulation, numerical computation, and visualization respectively.

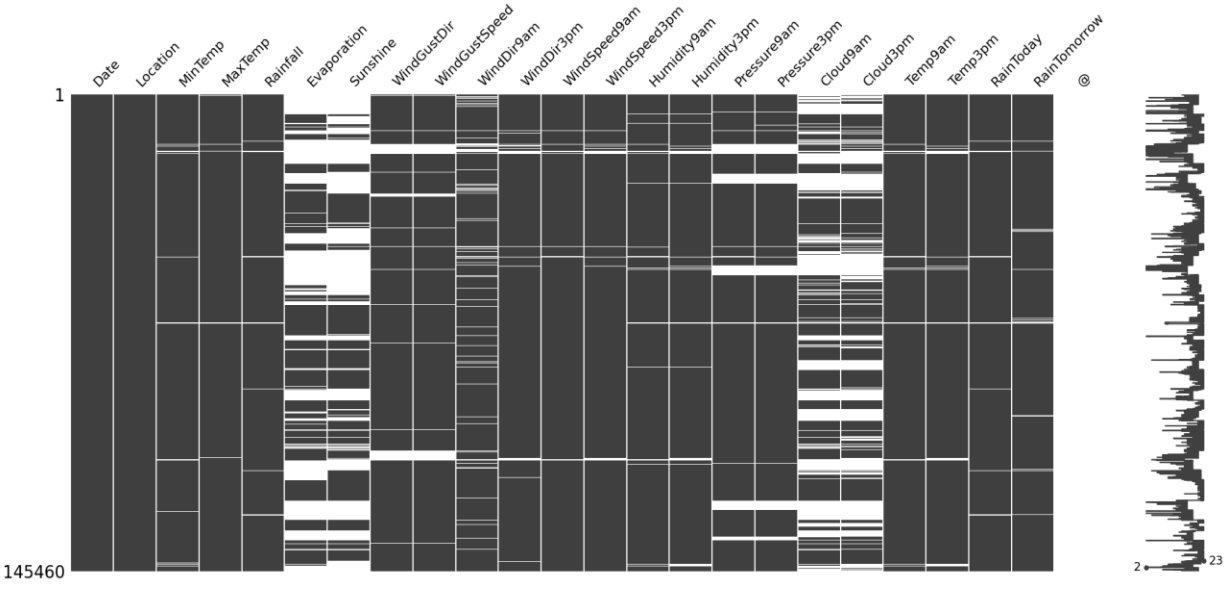
Handling Missing Values

After loading the dataset, missing values were identified and handled appropriately. For numerical columns, missing values were replaced with the mean, while for categorical columns, the most frequent category was used. A missing data heatmap was also visualized to identify null value patterns.

Missing Data Visualisation using Missingo

The figure represents the missing value matrix generated using the **Missingno** library in Python. This visualization provides a clear, structured overview of the distribution and density of missing data across all features in the rainfall dataset.

Each column represents a feature, and each horizontal line corresponds to a data record:

- Dark-colored blocks indicate the presence of data (non-missing values).
- White gaps represent missing (Nan) values.
- 

Encoding and Scaling

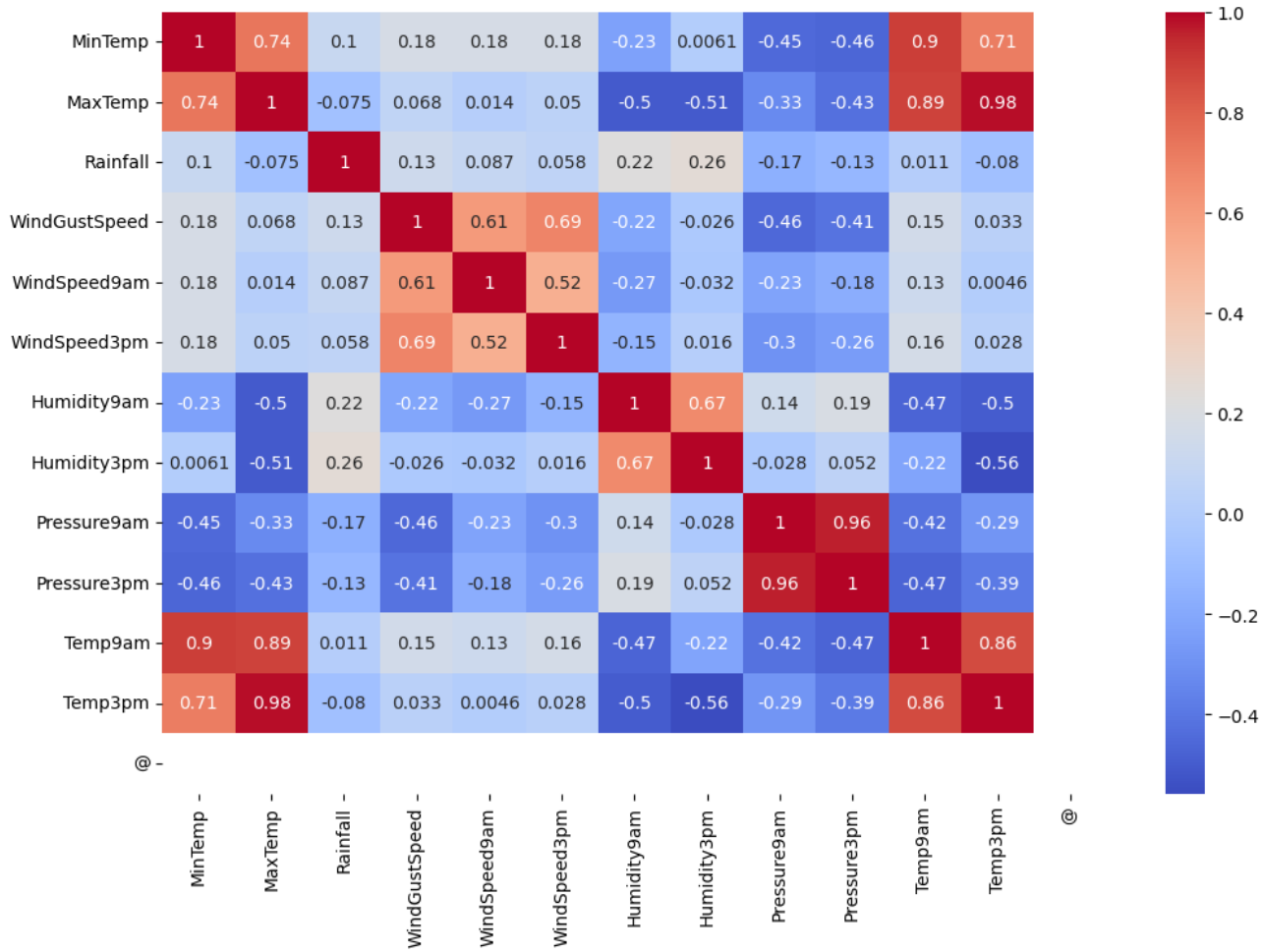
Categorical features were converted into numerical form using encoding techniques. Feature scaling was applied using StandardScaler to normalize numerical features, ensuring uniform contribution of each feature during analysis.

Data Visualization

Data visualization is the process of representing data in a graphical or pictorial format to make information easier to understand, interpret, and communicate. In the context of this project, it plays a vital role in exploring the rainfall dataset, identifying trends, correlations, and anomalies that may not be immediately apparent from raw numerical data.
Visualization helps transform complex datasets into meaningful insights by using charts, graphs, and plots

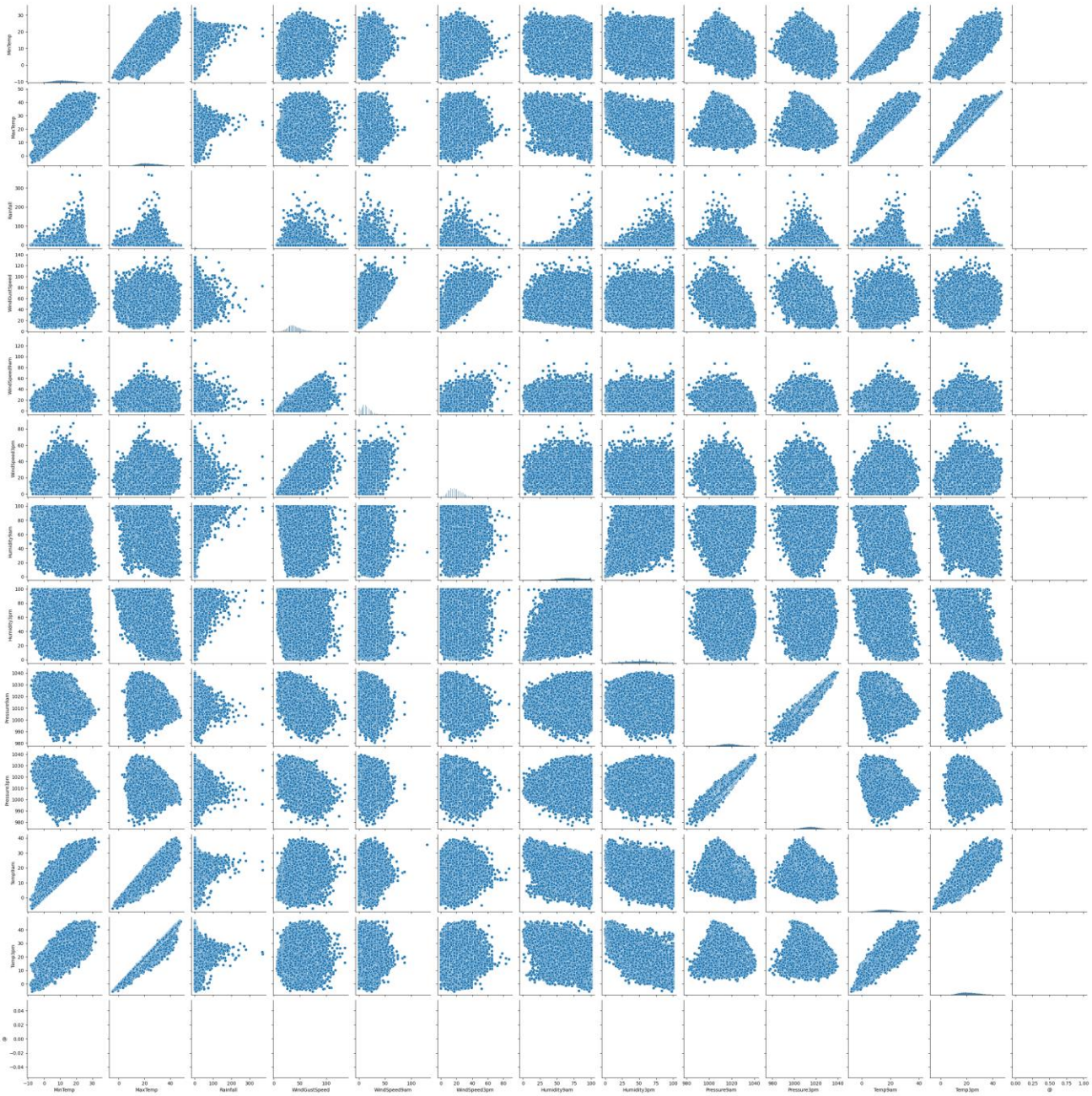
Correlation Heatmap

A heatmap was used to visualize the correlation between different numerical variables. It helped in identifying strong positive or negative relationships. Lighter shades indicated stronger correlations, while darker shades indicated weaker relationships.



Pair plot

The pairplot visualized pairwise relationships between numerical variables. It displayed scatterplots for variable combinations and histograms on the diagonal, helping to detect clusters, trends, and outliers.



Methodology

- Dataset: Questionnaire-based dataset with demographic and behavioural attributes.
- Data Preprocessing: Missing value handling, feature encoding, normalisation.
- Model Training: Logistic Regression / Random Forest trained in Jupyter Notebook.
- Model Persistence: Trained model exported as model_pkl.pkl.
- Evaluation: xgboost,regression used to validate the model.

Implementation

The system uses a Flask web application where users interact with an HTML/CSS/JS frontend questionnaire. The backend, managed by app.py, receives user inputs and sends them to the trained ML model (located in model/). The model performs the prediction, and the backend logic instantly processes the result. Finally, the system dynamically serves the appropriate HTML template (from templates/) to display the real-time prediction and relevant agricultural advice. Static assets like CSS and JavaScript are organized in the static/ directory.

Result

The project successfully implemented a Flask web application for rainfall prediction, primarily aiding agricultural decision-making. The core achievement was the trained Machine Learning model, with the Random Forest Classifier achieving the highest accuracy of 85.69% on the test dataset. The system dynamically renders advisory pages, displaying either chance.html or noChance.html, based on the model's prediction of rainfall. Necessary data preprocessing steps, including filling missing numeric values with the mean and categorical values with the most frequent strategy, were successfully applied. This architecture provides farmers with real-time advisories to optimize crop selection and irrigation schedules.

Future Scope

- The system can be utilized by policymakers and insurance agencies for effective agricultural risk assessment, including planning for droughts or floods.
- Findings can be leveraged by farmers to refine crop planning and selection, optimizing planting schedules based on local rainfall patterns.

- Agricultural experts can benefit by optimizing water usage and irrigation scheduling, developing efficient strategies to prevent waterlogging or drought damage.
- Future work includes enhancing predictive accuracy by exploring more sophisticated Ensemble Techniques and expanding the model's feature set

Demonstration

This section provides visual evidence of the developed project, including source code snippets and the web application interface.

Figure 1: app.py

app.py | rainfall_prediction.ipynb | index.html | nochance.html

app.py > predict

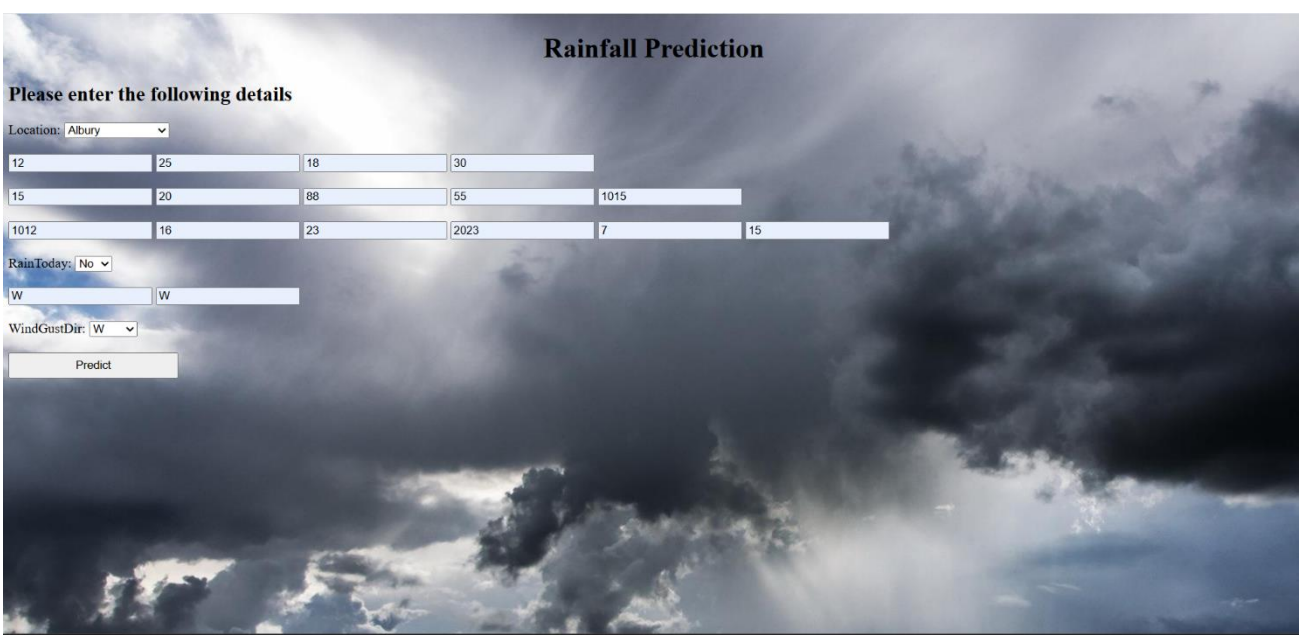
```python
import numpy as np
import pandas as pd
import joblib
from flask import Flask, request, render_template

app = Flask(__name__, template_folder='templates')

# Load trained model
model = joblib.load("rainfall.pkl")


# Home page
@app.route('/')
def home():
    return render_template('index.html')


# Prediction route
@app.route('/predict', methods=["POST"])
def predict():

    # Get form data
    data = request.form.to_dict()

    # Convert empty strings to NaN
    for key in data:
        if data[key] == "":
            data[key] = np.nan

    # Numeric columns (must match training dataset EXACTLY)
    numeric_cols = [
        'MinTemp','MaxTemp','Rainfall','WindGustSpeed',
        'WindSpeed9am','WindSpeed3pm',
        'Humidity9am','Humidity3pm',
        'Pressure9am','Pressure3pm',
        'Temp9am','Temp3pm',
        'Year','Month','Day'
```

app.py | rainfall_prediction.ipynb | index.html | nochance.html

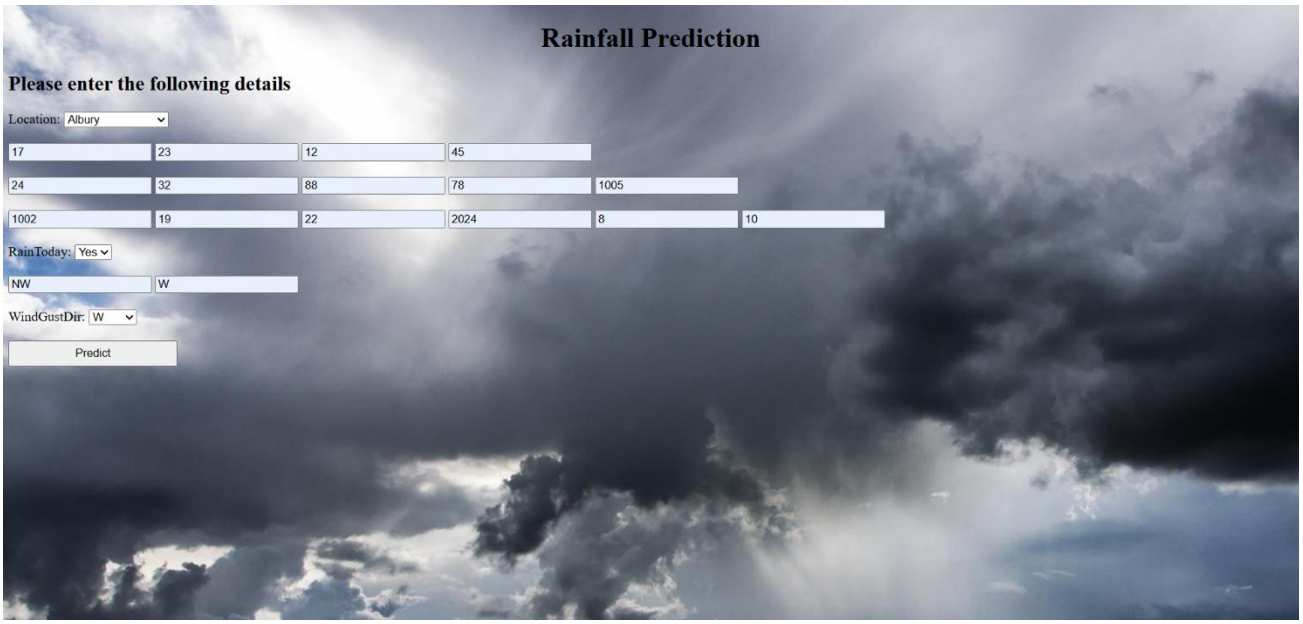
app.py > ...

```python
def predict():
        'Temp9am','Temp3pm',
        'Year','Month','Day'
    ]

    # Convert numeric columns to float
    for col in numeric_cols:
        if col in data and data[col] is not np.nan:
            try:
                data[col] = float(data[col])
            except:
                data[col] = np.nan

    # Convert dictionary to DataFrame
    df = pd.DataFrame([data])
    df['@'] = 0

    # Make prediction
    prediction = model.predict(df)[0]

    # Optional: Probability
    try:
        probability = model.predict_proba(df)[0][1] * 100
    except:
        probability = None

    # Render result page
    if prediction in ["Yes", 1]:
        return render_template("chance.html", prob=probability)
    else:
        return render_template("nochance.html", prob=probability)


if __name__ == "__main__":
    app.run(debug=True)
```

output



NO Chances of rain today.


Rainfall Prediction
Please enter the following details
Location: Albury
17
23
12
45
24
32
88
78
1005
1002
19
22
2024
8
10
RainToday: Yes
NW
W
WindGustDir: W
Predict

Conclusion

The exploratory analysis successfully provided valuable insights into rainfall patterns, trends, and variability across Indian regions. By applying data visualization techniques and implementing various Machine Learning algorithms (with Random Forest achieving the best performance), the project met its primary objective of creating a functional, predictive system. The resulting Flask web application offers a practical tool to deliver real-time rainfall predictions and tailored agricultural advisories, supporting better decision-making for farmers, agricultural experts, and policymakers in managing crop planning, irrigation, and agricultural risk assessment